CS 2212A Fall 2024 Group Project

Project Title: Alien Pal: Save & Care – Requirements Documentation

Table of Contents

1. Revision History

1.1 Version Date Author(s) Summary of Changes:

Version	Date	Author(s)	Summary of Change
1	October 6th	Whole Team	Table of contents, 2.1, 2.2, 2.3, 2.6 added
2	October 7th	Keefe Feng	Completed 2.2, 2.3, tried fixing the format of the documentation, team contract format
3	October 7th	Mike Tran	Table of contents created, and name of each section updated.
4	October 9th	Mike Tran	Completed the Summary with sections 4.1 - 4.4. Formatted the general looks of the document and updated table of contents.
5	October 9th	James Song	Completed "Overview" section of the introduction
6	October 9th	Keefe Feng Piercen Berardo James Song Devan Sodhi	Completed 2.4 (Functional Requirements), use case diagram, activity diagram. Completed the team contract, and submitted on GitLab
7	October 9th	Mike Tran	Formatted sections 4.1 - 4.5 and the general looks of the document. Final update to table of contents

2. Introduction

2.1 Overview:

Virtual pets are digital companions that simulate the experience of caring for a real pet. Players are responsible for taking care of it by performing an array of simple and repetitive tasks. These games provide a low-stake environment to manage daily tasks and routines related to pet care. This project will provide a program that simulates this type of software.

In this project, a main menu, tutorial, new game/pet selection, save/load game, and a gameplay screen are required. The player should also be able to have multiple pets save files and be able to switch between saves. During gameplay, the pet's vitals such as health, sleep (tiredness), fullness (hunger), and happiness should be displayed, and every vital stat except for health will deteriorate over time. Players will also be provided with commands that tell the pet to sleep, feed the pet, give the pet a gift, take it to the vet, play with the pet, and exercise with it (walks, hamster wheel, strength training etc.). There will be five states the pet can be in (dead, sleeping, angry, hungry, normal). When the pet is in the dead state or the sleeping state, the player cannot give it commands. When the pet is angry, the player can only give it gifts and play with it. For the rest of the states, the player can use any command. The player should also have an inventory of where food items and gift items will be stored. The pet's well-being and affection will be displayed as a score (one for each value). Each mood/state the pet can be in will be represented with an individual sprite. A series of parental controls will also be available such as setting a play time limit, keeping track of total play time, and being able to reset a pet on any save file back to the normal state. The program overall should exit cleanly and save data appropriately.

"Alien Pal: Save and Care" will simulate the experience of caring for a real pet. At the start of the game, players must choose one of three pets to save, where they will take home the one pet they decided to save. Players are then tasked with taking care of the pet by feeding it, grooming it, and playing with it. The pet can become agitated or sad due to bad care, so it is the player's responsibility to feed the pet daily, play with it often, and take care of its hygiene regularly. For example, not feeding your pet enough will cause your pet to become tired and unhappy, which leads to their well-being deteriorating and becoming unwilling to play with the player. Under good care, the pet's well-being will increase and will grow increasingly more attached to the player, which rewards the player for completing positive, yet simple and repetitive tasks. When the pet reaches a certain affection level, they offer the player gifts which are collectibles that mark the progress of the player and the pet.

2.2 Objectives:

The primary focus of this project is to fill the niche of virtual pets, by creating an engaging and interactive experience for our users. As the gaming industry continues to undergo constant development, our aim for this project is to provide a unique sense of emotional engagement, nostalgia, and entertainment.

By focusing on creating a virtual pet simulator, this project aims to:

1. Appeal to a wide demographic - A game that attracts players of various ages and interests, from children and teens who enjoy playful, casual gaming to adults seeking nostalgic or low-maintenance virtual companionship.

2. Develop a sense of companionship - Designed to nurture an emotional connection between the player and their pet. By incorporating realistic behaviors and evolving relationships, the game will encapsulate the experience of caring for a real pet. Through daily care tasks, responsive behaviors, and the pet's ability to express emotions such as happiness, sadness, or anger, users will feel a deep sense of attachment and responsibility toward their virtual companion.

3. Create a long-term engaging form of entertainment - The game will include features that encourage players to keep coming back, such as daily tasks, rewards, and pet growth over time. Simple mechanics like feeding, playing, and taking care of the pet will help maintain player interest and make the experience enjoyable over the long term.

4. Fill the virtual pet niche – As virtual pet games become rarer, this project aims to revive and expand the genre by offering players an experience of companionship, interaction, and entertainment through unique features and mechanics.

The main objectives of the game are to:

1. Ensure long-term player engagement – Through daily activities, evolving gameplay mechanics, and a pet growth system, the game will offer continuous entertainment and a reason for players to return regularly.

2. Offer simple yet rewarding gameplay – By focusing on straightforward and intuitive mechanics such as feeding, playing, and caring for pets, players will find the game easy to pick up while still providing depth for long-term enjoyment.

3. Provide players with an emotional connection – By designing pets that display realistic behaviors and emotions, players will form a meaningful bond with their virtual companions, creating a rewarding and emotionally engaging experience.

2.3 References:

No external references were used for this document. All content is original and based on the project specifications.

3. Domain Analysis

The software domain will be game development:

- The target audience is teenagers and young adults, specifically people who like light-hearted interactive games.
- This game will specifically be in the subdomain of casual games, more specifically virtual pet simulation.
- This domain focuses on user engagement and repeatability. Players will form an emotional attachment to their pet and get immersed in keeping the pet happy and healthy. Players will form daily habits of "checking in" on their pets, so repeatability must be a strong virtue in game development.
- Common issues:
    1. If the game is not repayable, it can get quite boring and ruin the immersive experience.
    2. If players run out of things to do in the game, they may get bored and stop playing.
    3. On the other hand, if the game is too complex and has a high learning curve, players may be hesitant to play. Players are looking for an easy and light-hearted game with lots of replay ability.
- Common solutions:
    4. Creating tasks and challenges that appear throughout the game can help retain user satisfaction and avoid users from getting bored. Over time, new challenges will be unlocked, which could lead to rewards.
    5. Rewards keep users engaged. Rewards from challenges or keeping the users pet happy and healthy will influence users to repeatably come back to the game.
    6. If there are rewards but no progression, the game can get boring, so progression is important. Pets should be able to get upgrades, or the user should get better equipment to play with their pet.
- We will use this knowledge to make a game that is engaging, has reliability, and has a very thought-out design and story to keep the user engaged. We will implement challenges and progression in the game to overcome these common issues.

4. Functional Requirements

4.1 User Interface:

- Multiple pages: main menu, gameplay, tutorial, load/new game, parental controls
- Mouse-Based Interaction Support
- Keyboard Shortcuts: commands for feeding, opening inventory, etc.
- Pet feedback: The pet will play an animation depending on its mood and hunger, and when you perform certain actions.

❖ Main Menu:
- ➢ Title
- ➢ New game
- ➢ Load Game
- ➢ Settings:
  - ▪ Tutorial
  - ▪ Parental controls
- ➢ Exit

❖ Instructions / Tutorial:
- ➢ One screen with detailed instructions on how to play the game
- ➢ Should document all major features from the player perspective

❖ New game & Pet Selection:
- ➢ Player is presented with opening scene and pet options with basic information about the pets
- ➢ Players can name the pet
- ➢ Brought to main game screen

❖ Save/Load Game:
- ➢ Saves/Loads game state (pet information) locally.
- ➢ Make it so the player can resume their progress from their last play session
- ➢ A clear confirmation message should be displayed each time the game state is saved
- ➢ Must be designed such that the player can have multiple pets that they can switch between by saving and then loading a new pet

❖ Vital Statistics & Rules:

- Health: A positive numerical value that indicates the pet's health. If the value reaches zero, then the pet is dead.
- Sleep: A low positive value indicates that the pet is sleepy, a positive high value indicates they are fully awake.
- Fullness: A low positive value indicates that the pet is starving, a high positive value indicates that they are full.

- These statistics will be shown by numerical values and pictorially. All statistics except health will slowly decline at a set rate over time. When a statistic is less than 25% of the maximum value, there will be a flashing text hovering over the statistic. When a statistic reaches zero, the following will happen:
  - Health: The pet will die and will be buried in the backyard. The game will be over and will give the player the option to start a new game or load a game.
  - Sleep: A set number of health points will be removed and then the pet will immediately fall asleep. They will sleep until the statistic reaches the maximum value. When the max is reached, they will return back
  - Fullness: The pet will enter the starvation state, a set number of happiness points will be removed. Health points will continue to decrease until it eats food, and the statistic is above zero.

- Happiness: A low positive value indicates that the pet is depressed, a high positive value indicates that they are joyful.
  - The pet will enter a depressed state, they will not listen to any commands except activities that increase happiness and will only return to normal when the statistic is over ½ of the maximum value. If left without care, the pet will be constantly thinking that the user will throw them away at any moment. At random, they will leave the house and never be found again, the user will be given the option to start a new game or load a file.
  - The pet sprite will change to the state that it is in, and all vital statistics will be saved when the game is saved.

- Commands:
  - Go to bed
  - Feed
  - Give Gift
  - Take to the Vet
  - Play
  - Exercise

- Based on the pet's current state, the following commands are available to the player:
  - Dead State
  - Sleeping State
  - Angry State
  - Hungry State
  - Normal State

- Player Inventory:
  - Food Items
  - Gift Items
  - Keeping Score

- Pet Sprite:
  - The pet sprite will be created by the team.

- Parental Controls:
  - Parental Limitations
  - Parental Statistics
  - Revive Pet

- Housekeeping & error handling

- Add progression system & more items, currency, and an item store:
  - A currency will be rewarded to the player for completing certain tasks and keeping their pet happy and healthy.
  - This currency can be used to purchase upgrades, food, gifts, etc. for the user's pet.

4.2 Actors:

Actor Name	Child User
Description	This actor is the user of the software. They can start a new game, interact with their virtual pet, and change the general settings.
Aliases	Player
Inherits	None
Actor Type	Human
Active/Passive	Active

Actor Name	Parent User
Description	This actor is the parent of the player. They can do anything the user can do and change the parental controls.
Aliases	Parent
Inherits	Child User
Actor Type	Human
Active/Passive	Active

4.3 Use Cases:

Name	Tutorial
Actor	Child User
Secondary Actor	None
Goal in context	To view the tutorial for the game
Preconditions	The user is in the main menu
Trigger	When the user clicks the “Tutorial” button
Scenario	The user clicks the “Tutorial” button, then the tutorial (whether it be a video or something else), opens. The user has the option to exit the tutorial
Exceptions	Tutorial not displaying properly
Priority	Lowest priority

Name	Save/Load game
Actor	Child User
Goal in context	To overwrite a pre-existing save file (save), or use a pre-existing save file to update the current game data (load).
Preconditions	The user must be in the main menu to load a game, and the user must already have a save file and be in the game to save the game.
Trigger	“Save” or “Continue” buttons are pressed
Scenario	The user presses the “Save” button and the current game data file is overwritten with the current game data. When the user presses the “Continue” button the game is loaded using data provided from file on the user's system stored locally.
Exceptions	The user accidentally saves the game and overwrites their file. This can be circumvented by implementing a confirmation screen to save your game. The user accidentally loads their game when they want to start a new one. This can be corrected by allowing the user to go back to the menu from inside the game. The user exits without saving their game. This can be circumvented by storing the unsaved game state in a different file and asking the user if they want to save it or use it next time, they login. If the user exits by going to the main menu and doesn’t save, we can prompt them to save their game or continue to the main menu.
Priority	High

Name	Start new game
Primary Actor	Child User
Secondary Actor	None
Goal in context	To create a new save file and start a new game
Preconditions	The user is in the main menu
Trigger	When the user clicks the “start new game” button
Scenario	The user clicks the “start new game” button, then a new save file is created. The user is then presented with the opening scene of the game.
Exceptions	The user could potentially press “start new game” multiple times, which could cause bugs. To mitigate this, once the “start new game” button is clicked, the button will disappear and be replaced with a loading symbol.
Priority	Highest priority

Name	Exit
Primary Actor	Child User
Secondary Actor	None
Goal in context	To Exit the game and save the game state
Preconditions	The user must be in the main menu
Trigger	“Exit” button pressed
Scenario	The user presses the “Exit” button, and the game calls the save game state use case and closes the game.
Exceptions	The game crashes before saving
Priority	Medium

Name	Pet Selection
Primary Actor	Child User
Secondary Actor	None
Goal in Context	The user is provided with 3 aliens and selects which pet they want to take care of.
Preconditions	The user has selected the “start new game” button in the main menu. A new save file has been created and initialized with the correct information.
Trigger	Start new game process finished
Scenario	The opening scene is played, and the user is provided with the option of 3 pets. There is a button under each pet that says, “select pet”. Once clicked, the user can name the pet and start a new game. If they want to go back, there is a back button.
Alternatives	None
Exceptions	The file does not initialize with the correct information.
Priority	Highest

Name	Vital Statistics
Primary Actor	Child User
Secondary Actor	None
Goal in Context	Display clearly the pets' current vitals such as its health, sleep, fullness, and happiness.
Preconditions	The user must be playing the game and have a pet.
Trigger	User is loaded into a game.
Scenario	The user loads a new game or a save file and is on the main pet screen. They are able to see their pet’s health, sleep, fullness, and happiness.
Alternatives	The user is on another screen such as the instructions or settings.
Exceptions	The data file is altered, and it breaks the program. In this case it should be caught, and the user should be prompted with a message saying the save file is corrupted and to create a new game.
Priority	High

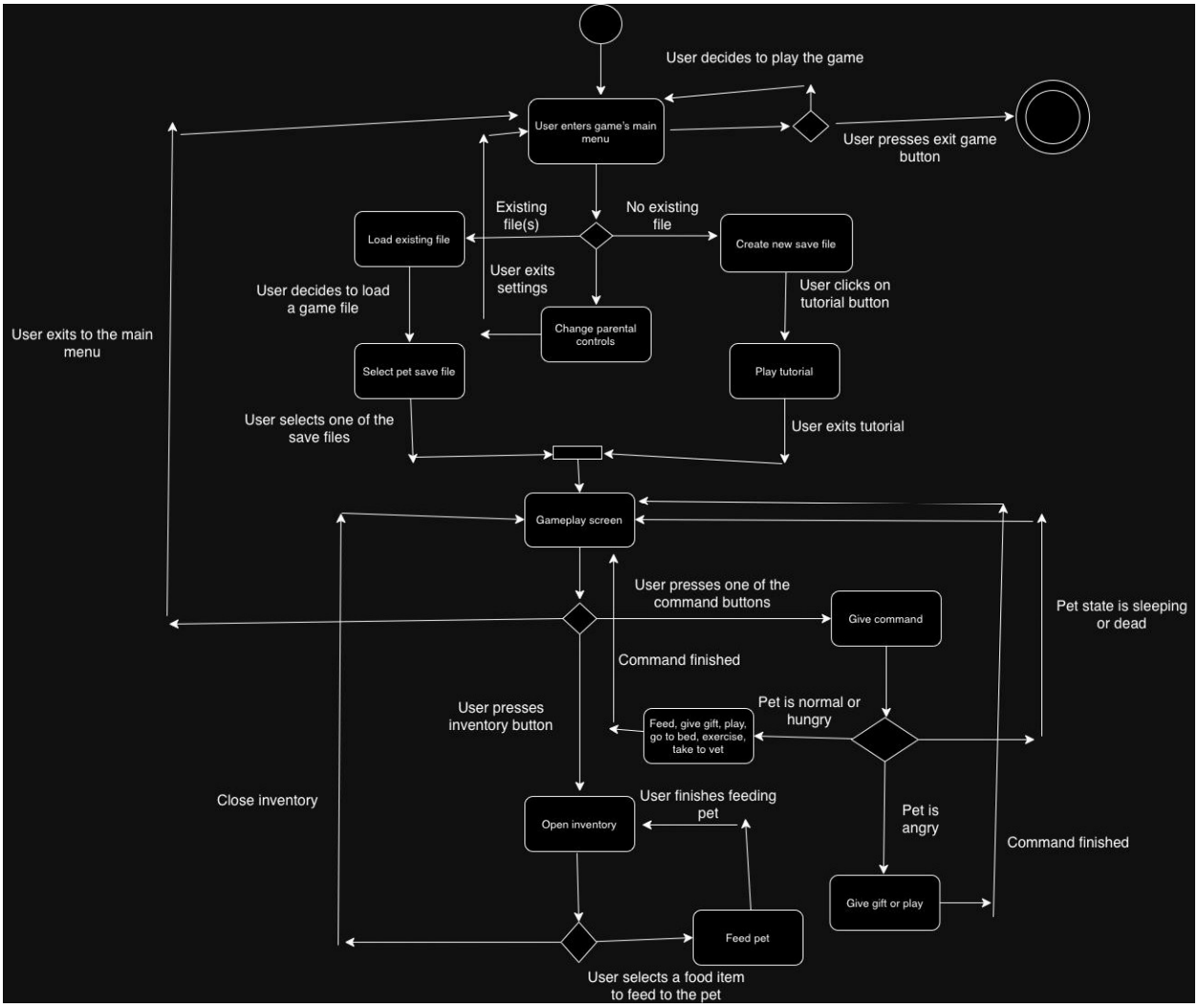
Name	Command the pet to do something
Primary Actor	Child user
Secondary Actor	None
Goal in Context	To interact with the virtual pet and progress through the game
Preconditions	The user must be in the gameplay screen
Trigger	The user clicks on one of the command buttons
Scenario	The user clicks on the “Go to bed” button, and the pet goes to sleep. The player must wait for the pet to finish sleeping before issuing another command. When the pet wakes up, the user presses “Play with pet” button, where the user raises the well-being level of the pet.
Alternatives	The user can press the “Feed”, “Give gift”, “Take to the vet”, or “Exercise” command button to potentially raise their pet’s well-being. If the pet is in the Sleep state or the Dead state, no commands can be issued. If the pet is in the Angry state, only the “Give gift” and “Play” commands will be available.
Exceptions	The user might be able to click multiple commands at once. To prevent this, there will be a short period of time after the user has clicked a command button where they can’t click another command.
Priority	Medium

Name	Player Inventory
Primary Actor	Child user
Secondary Actor	None
Goal in Context	To open the inventory menu
Preconditions	The user must be in the game menu
Trigger	The user selects the “inventory” button
Scenario	The user selects the “inventory” button, and the inventory menu opens. They have the option to select “food items” or “other items”. Upon selection, the respective inventory is displayed. The user can also exit the inventory.
Alternatives	

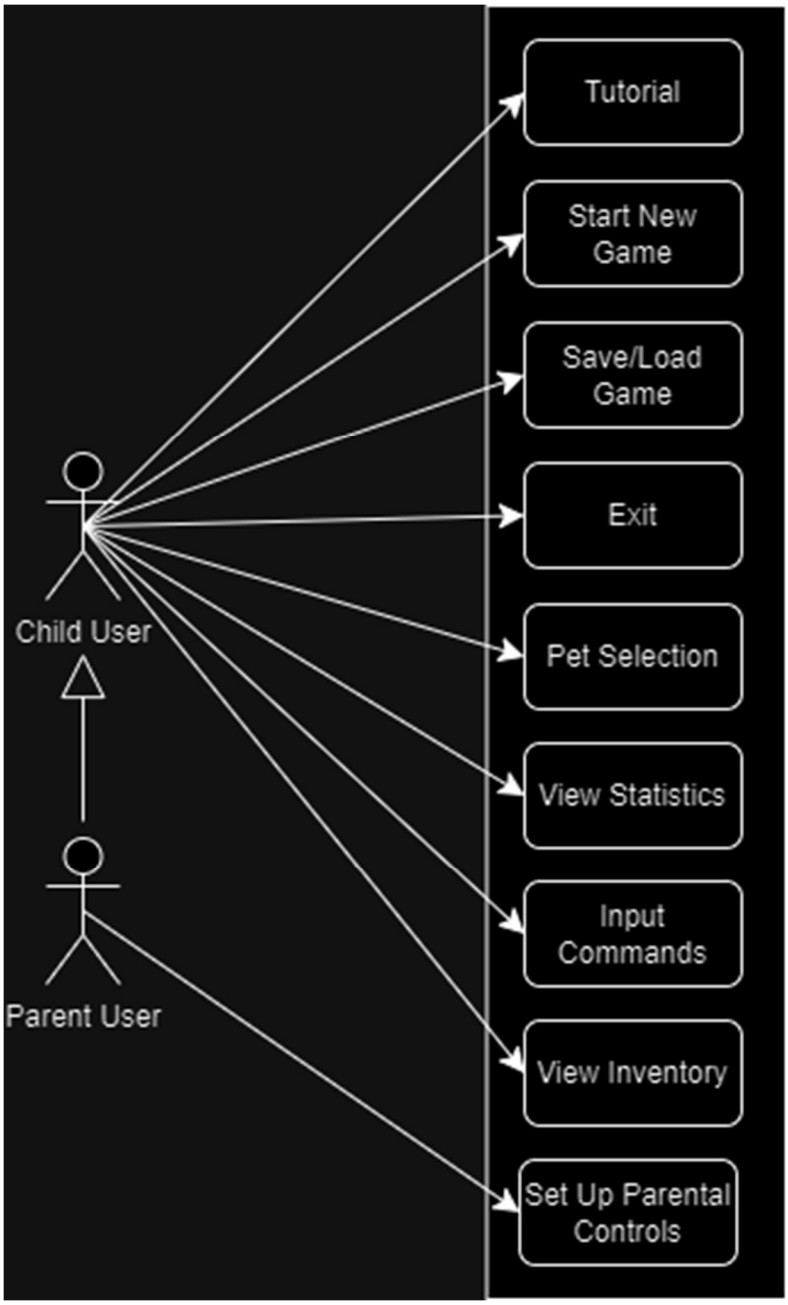
Exceptions	The user clicks the inventory button too many times and opens multiple instances
Priority	High

Name	Parental Controls
Primary Actor	Parent User
Secondary Actor	None
Goal in Context	Set restrictions, view statistic, and allow for certain “cheats”.
Preconditions	The parental controls setting must be properly set up with a password.
Trigger	The user enters settings and presses the parental controls option.
Scenario	A player's pet dies, the parent enters the parental controls and revives the pet allowing the child to continue playing.
Alternatives	None
Exceptions	The parent password is entered wrong. In this case the user will not be able to access parental control until the password is entered correctly.
Priority	Medium

4.4 Activity Diagram:



4.5 Use Case Diagram:



5. Non-functional Requirements

1. The programming language used must be Java 23 or newer to ensure its compatible with newer versions of Java.
2. The game must have an object-oriented design and principles, which should be explained in the project documentation.
3. The game must have a user-friendly GUI. The interface should be intuitive, cursor-based, and easy to use for users aged 7+.
4. All game data should be stored locally without the use of an internet connection, for example, a JSON file.
5. The game should only use freely available libraries that are easy to obtain and install.
6. All code and project files should be maintained in the GitLab repository. This includes proper documentation and issue tracking throughout the project's development.
7. All code must be documented using Javadoc. Each method and class should have a corresponding comment explaining its purpose.
8. Most of the code must be unit-tested using JUnit 5. GUI elements are exempt but should be tested through other means if applicable.
9. The application must be executable on Windows 11 systems with a standard Java installation (Java 23 or newer).
10. The game must not alter or delete files outside its own directory. All saved files and data must be contained within the application's local directory.
11. The application must display clear error messages for invalid user inputs and other errors. Additionally, it should provide immediate visual feedback for all user actions.
12. The application should be optimized for efficient performance, using minimal system resources while maintaining responsiveness during gameplay.
13. The interface should be accessible, offering both mouse and keyboard controls, ensuring that color usage is appropriate for color-blind players, and maintaining logical tab orders for navigation.
14. Code must be structured to be maintainable and reusable, supporting future updates and modifications without requiring extensive rewrites.
15. The complete project, including all assets and dependencies, must remain under 500MB.
16. All content, comments, and documentation must be in English and adhere to standard software engineering principles.

6. Summary

The **"Alien Pal: Save & Care"** project outlines the development of a virtual pet simulator where players care for an alien pet by feeding, playing with, and tending to its well-being. The game focuses on creating an emotional connection between players and their pets, fostering long-term engagement through daily tasks, rewards, and progression.

6.1 Key Features:

- **Main Gameplay**: Players choose from three alien pets, each requiring regular care (feeding, playing, hygiene). Pets' express emotions like happiness, sadness, or anger based on the player's care.
- **Pet States**: Pets can be in five distinct states—dead, sleeping, sad, hungry, or normal—impacting the player's ability to interact with them.
- **Inventory System**: Players store food and items to use for pet care.

6.2 Objectives:

1. Create a game that appeals to a wide demographic (children to adults).
2. Nurture companionship by making pets responsive and emotionally expressive.
3. Ensure long-term engagement through tasks, rewards, and pet growth.
4. Revive the virtual pet genre by offering unique, interactive features.

6.3 Domain and Challenges:

The game belongs to the **casual virtual pet simulation** genre, targeting teens and young adults. Key challenges include maintaining player interest through replayability, providing progression, and balancing simplicity with depth to avoid boredom or high complexity.

6.4 Non-functional Requirements:

- The game must be developed in **Java 23** or newer, adhering to object-oriented principles.
- It will feature a **user-friendly GUI** for players aged 7+, with local data storage and minimal system resource usage.
- The project enforces **good coding practices**, including Javadoc documentation, JUnit 5 testing, and maintaining code in a **GitLab repository**.
- **Accessibility** features will be implemented, supporting both mouse and keyboard controls, along with a colorblind-friendly design.